



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 18

“Pantalla LCD 16x2”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 18 de marzo de 2026

INTRODUCCIÓN

Pantalla LCD 16x2

Las pantallas LCD 16x2 (Liquid Crystal Display) son el estándar de oro para la visualización de texto en sistemas embebidos. El nombre "16x2" indica su capacidad: puede mostrar 16 caracteres por fila en 2 filas, para un total de 32 caracteres simultáneos.

La gran mayoría de estas pantallas se basan en el controlador industrial Hitachi HD44780, lo que significa que casi cualquier microcontrolador (incluyendo tu ATmega8535) tiene librerías compatibles para manejarlas.

Arquitectura y Funcionamiento

A diferencia de una matriz de LEDs, donde tú controlas cada punto, el LCD tiene un controlador interno que ya sabe cómo dibujar las letras. Tú solo le envías el código ASCII (por ejemplo, el número 65 para la "A") y él se encarga de encender los cristales líquidos correspondientes.

Los Cristales Líquidos

Cada carácter está formado por una matriz de 5x8 píxeles. Cuando se aplica un voltaje a una celda, las moléculas de cristal líquido se alinean para bloquear o permitir el paso de la luz (proveniente del *Backlight*), creando el contraste que vemos como una letra.

Descripción de los Pines (Interfaz de 16 pines)

Casi todas las pantallas 16x2 comparten la misma distribución de pines:

- **VSS / VDD / V0:** Alimentación (GND/5V) y el pin de Contraste. Este último se conecta a un potenciómetro para ajustar qué tan legibles son las letras.
- **RS (Register Select):** Indica si el dato enviado es un comando (como "limpiar pantalla") o un carácter para escribir.
- **RW (Read/Write):** Define si vas a escribir en la pantalla o leer de ella (casi siempre se deja en GND para escribir).
- **E (Enable):** Es el pin que "confirma" el envío de datos. Sin un pulso en este pin, la pantalla ignora las señales.
- **D0-D7:** Bus de datos de 8 bits.
- **A / K (Anode/Cathode):** Alimentación para el LED de luz de fondo (Backlight).

Modos de Operación: 8 bits contra 4 bits

Esta es una característica clave para ahorrar pines en un microcontrolador:

- **Modo de 8 bits:** Utiliza todos los pines de datos (D0-D7). Es el más rápido, pero consume muchos pines del microcontrolador.

- **Modo de 4 bits:** Solo utiliza los pines D4 a D7. El microcontrolador envía cada byte en dos mordiscos (nibbles) de 4 bits. Es el estándar en la mayoría de los proyectos porque ahorra 4 pines valiosos sin una pérdida de velocidad perceptible para el ojo humano.

CGRAM: Caracteres Personalizados

Una función avanzada y muy útil es la CGRAM (Character Generator RAM). Permite al usuario diseñar hasta 8 caracteres propios que no están en la memoria de fábrica. Esto es ideal para crear símbolos de batería, iconos de temperatura, barras de progreso o incluso letras con acentos (que no siempre vienen por defecto).

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 18, el circuito a armar es el siguiente:

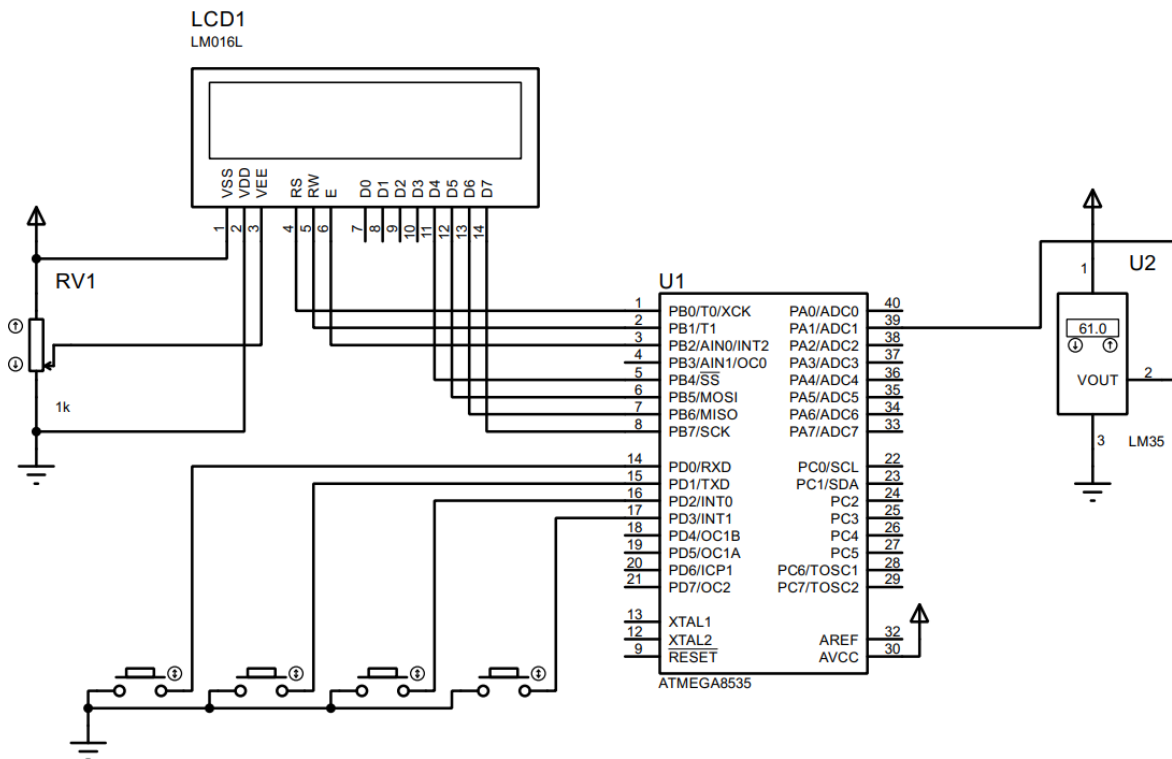


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen, registro B se configura como de salida (PB0-PB2, PB4-PB7), mientras que el registro D como de entrada (PD0-PD3) activando las resistencias de pull-up.

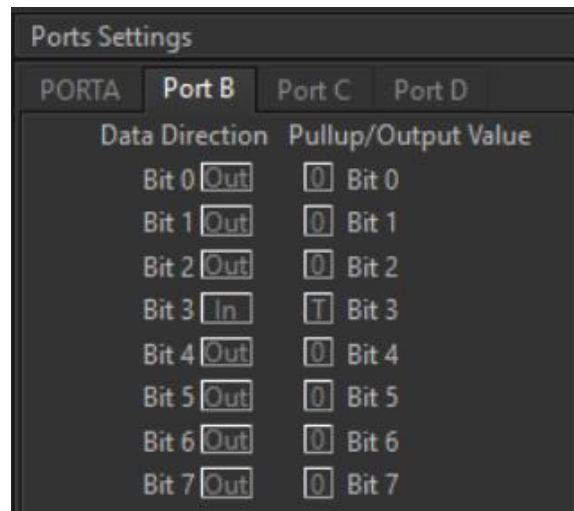


Imagen 2. Configuración del puerto B.

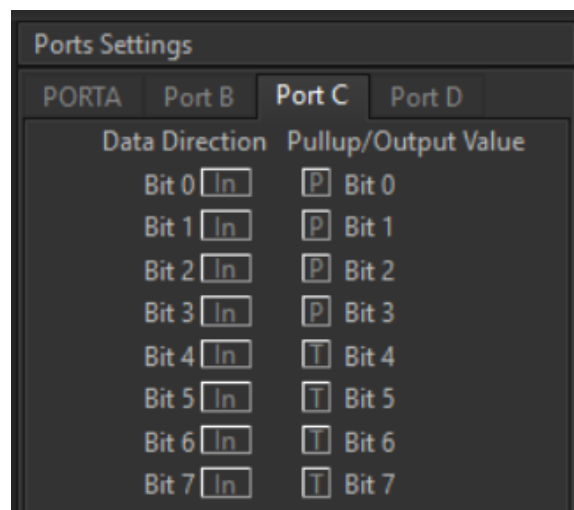


Imagen 3. Configuración del puerto C.

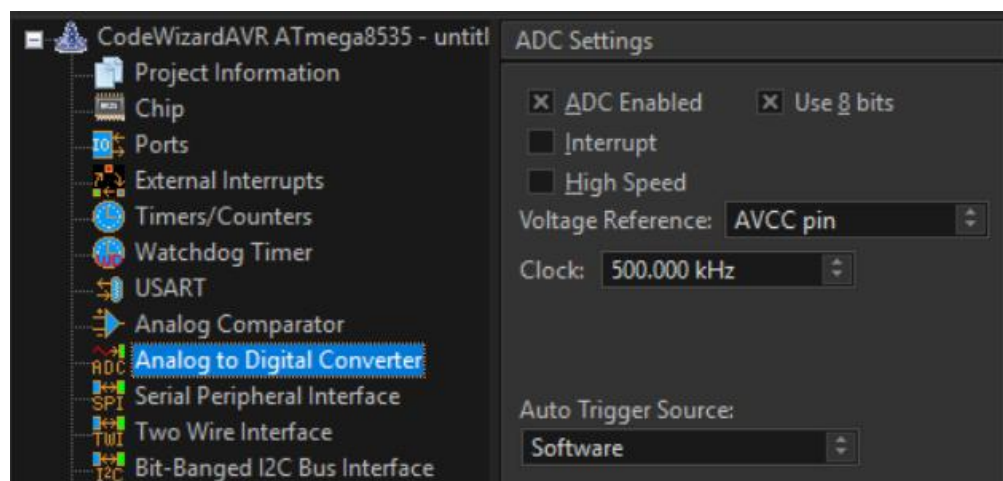


Imagen 4. Configuración del convertidor analógico digital.

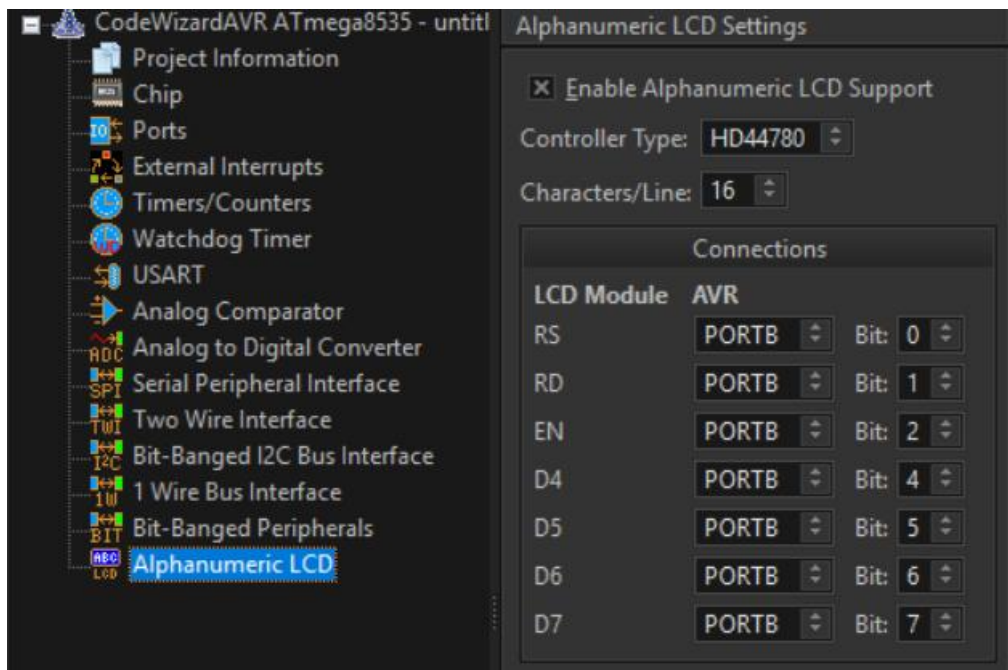


Imagen 5. Configuración de los pines para el módulo LCD.

Código del circuito de CodeVision:

```
1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : PrÃ¡ctica 18 'Pantalla LCD 16x2'
8  Version : 1.0
9  Date    : 14/03/2026
10 Author  : GarcÃ-a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model        : Small
19 External RAM size    : 0
20 Data Stack size      : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25
26 // Delay functions
27 #include <delay.h>
```

```
29 // Alphanumeric LCD functions
30 #include <alcd.h>
31
32 #define BTN_CAMBIAR_MODO PIND.0
33 #define BTN_AJUSTAR_HORA PIND.1
34 #define BTN_AJUSTAR_MIN PIND.2
35 #define BTN_AJUSTAR_SEG PIND.3
36
37 // Declare your global variables here
38
39 // Variables de temperatura
40 float temperatura_celsius;
41 int temperatura_entera;
42
43 // Control de tiempo del programa
44 int contador_reloj, contador_antirebote, tiempo_antirebote;
45
46 // Estados de botones
47 bit btn_presionado_anterior, btn_presionado_actual;
48
49 // Variables para mostrar temperatura
50 unsigned char temp_unidades, temp_decenas, temp_decimas, lectura_adc;
51
52 // Variables de reloj
53 unsigned char segundos = 0, minutos = 0, horas = 0;
54
55 // Variables de fecha
56 unsigned char dia_actual = 22, mes_actual = 8;
57
58 // Variables de aÃ±o
59 unsigned short siglo = 19, anio = 95;
```



```
61 // Control de modo (editar hora / editar fecha)
62 unsigned char modo_edicion;
63
64 const char ASCII_CERO = 48; //codigo ascii
65
66 // ADC Voltage Reference: AVCC pin
67 #define ADC_VREF_TYPE ((0<<REFS1) | (1<<REFS0) | (1<<ADLAR))
68
69 // Read the 8 most significant bits
70 // of the AD conversion result
71 // Read Voltage=read_adc*(Vref/256.0)
72 unsigned char read_adc(unsigned char adc_input) {
73     ADMUX=adc_input | ADC_VREF_TYPE;
74     // Delay needed for the stabilization of the ADC input voltage
75     delay_us(10);
76     // Start the AD conversion
77     ADCSRA|=(1<<ADSC);
78     // Wait for the AD conversion to complete
79     while ((ADCSRA & (1<<ADIF))==0);
80     ADCSRA|=(1<<ADIF);
81     return ADCH;
82 }
```

```
84 unsigned char leer_adc_promedio(unsigned char canal) {
85     unsigned int suma = 0;
86     unsigned char i;
87
88     for(i = 0; i < 20; i++)
89     {
90         suma += read_adc(canal);
91         delay_ms(2);
92     }
93
94     return suma / 20;
95 }
```



```

97 void main(void) {
98     // Declare your local variables here
99
100    // Input/Output Ports initialization
101    // Port A initialization
102    // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
103    DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
104    // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
105    PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
106
107    // Port B initialization
108    // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=In Bit2=Out Bit1=Out Bit0=Out
109    DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (0<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
110    // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=T Bit2=0 Bit1=0 Bit0=0
111    PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
112
113    // Port C initialization
114    // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
115    DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
116    // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
117    PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
118
119    // Port D initialization
120    // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
121    DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
122    // State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=P Bit2=P Bit1=P Bit0=P
123    PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);

125    // Timer/Counter 0 initialization
126    // Clock source: System Clock
127    // Clock value: Timer 0 Stopped
128    // Mode: Normal top=0xFF
129    // OC0 output: Disconnected
130    TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
131    TCNT0=0x00;
132    OCR0=0x00;

134    // Timer/Counter 1 initialization
135    // Clock source: System Clock
136    // Clock value: Timer1 Stopped
137    // Mode: Normal top=0xFFFF
138    // OC1A output: Disconnected
139    // OC1B output: Disconnected
140    // Noise Canceler: Off
141    // Input Capture on Falling Edge
142    // Timer1 Overflow Interrupt: Off
143    // Input Capture Interrupt: Off
144    // Compare A Match Interrupt: Off
145    // Compare B Match Interrupt: Off
146    TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
147    TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
148    TCNT1H=0x00;
149    TCNT1L=0x00;
150    ICR1H=0x00;
151    ICR1L=0x00;
152    OCR1AH=0x00;
153    OCR1AL=0x00;
154    OCR1BH=0x00;
155    OCR1BL=0x00;

```

```

157 // Timer/Counter 2 initialization
158 // Clock source: System Clock
159 // Clock value: Timer2 Stopped
160 // Mode: Normal top=0xFF
161 // OC2 output: Disconnected
162 ASSR=0<<AS2;
163 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
164 TCNT2=0x00;
165 OCR2=0x00;
166
167 // Timer(s)/Counter(s) Interrupt(s) initialization
168 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
169
170 // External Interrupt(s) initialization
171 // INT0: Off
172 // INT1: Off
173 // INT2: Off
174 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
175 MCUCSR=(0<<ISC2);
176
177 // USART initialization
178 // USART disabled
179 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

181 // Analog Comparator initialization
182 // Analog Comparator: Off
183 // The Analog Comparator's positive input is
184 // connected to the AIN0 pin
185 // The Analog Comparator's negative input is
186 // connected to the AIN1 pin
187 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
188
189 // ADC initialization
190 // ADC Clock frequency: 500.000 kHz
191 // ADC High Speed Mode: Off
192 // ADC Auto Trigger Source: Software
193 // Only the 8 most significant bits of
194 // the AD conversion result are used
195 ADMUX=ADC_VREF_TYPE;
196 ADCSRA=(1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (1<<ADPS0);
197 SFIOR=(1<<ADHSM) | (0<<ADTS2) | (0<<ADTS1) | (0<<ADTS0);
198
199 // SPI initialization
200 // SPI disabled
201 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
202
203 // TWI initialization
204 // TWI disabled
205 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

```

208 // Connections are specified in the
209 // Project/Configure/C Compiler/Libraries/Alphanumeric LCD menu:
210 // RS: PORTB Bit 0
211 // RD: PORTB Bit 1
212 // EN: PORTB Bit 2
213 // D4: PORTB Bit 4
214 // D5: PORTB Bit 5
215 // D6: PORTB Bit 6
216 // D7: PORTB Bit 7
217 // Characters/line: 16
218 lcd_init(16);
219 contador_reloj = 0;
220 contador_antirebote = 0;
221 tiempo_antirebote = 20;

```

```
223 while (1) {
224     delay_ms(1);
225
226     if (!BTN_CAMBIAR_MODO) {
227         btn_presionado_actual = 0;
228     } else {
229         btn_presionado_actual = 1;
230     }
231
232     if (btn_presionado_anterior && !btn_presionado_actual) {
233         if (!modo_edicion) {
234             modo_edicion = 1;
235         } else {
236             modo_edicion = 0;
237         }
238     }
239     btn_presionado_anterior = btn_presionado_actual;
240
241     // Se coloca la palabra 'ESCOM' en la LCD
242     lcd_gotoxy(11, 0);
243     lcd_putsf("ESCOM");
244
245     // Se coloca la temperatura
246     lectura_adc = leer_adc_promedio(1); // Leemos la temperatura
247     temperatura_celsius = lectura_adc * 1.95;
248
249     if (temperatura_celsius > 99) {
250         temperatura_celsius = 99;
251     }
```

```
253 // Dividimos y obtenemos el valor de la temperatura
254 temperatura_entera = temperatura_celsius * 10;
255 temp_decenas = temperatura_entera / 100;
256 temperatura_entera %= 100;
257 temp_decimas = temperatura_entera % 10;
258 temp_unidades = temperatura_entera / 10;
259
260 // Colocamos la temperatura en la LCD
261 lcd_gotoxy(10, 1);
262 lcd_putchar(temp_decenas + ASCII_CERO);
263 lcd_gotoxy(11, 1);
264 lcd_putchar(temp_unidades + ASCII_CERO);
265 lcd_gotoxy(12, 1);
266 lcd_putchar('.');
267 lcd_gotoxy(13, 1);
268 lcd_putchar(temp_decimas + ASCII_CERO);
269 lcd_gotoxy(14, 1);
270 lcd_putchar(ASCII_CERO + 175); // Car3cter Â°
271 lcd_gotoxy(15, 1);
272 lcd_putchar('C');
```

```
274 // Reloj en movimiento
275 if (modo_edicion) {
276     if (!BTN_AJUSTAR_HORA) {
277         if (contador_antirebote > tiempo_antirebote) {
278             contador_antirebote = 0;
279             horas++;
280         } else {
281             contador_antirebote++;
282         }
283     }
284
285     if (!BTN_AJUSTAR_MIN) {
286         if (contador_antirebote > tiempo_antirebote) {
287             contador_antirebote = 0;
288             minutos++;
289         } else {
290             contador_antirebote++;
291         }
292     }
293
294     if (!BTN_AJUSTAR_SEG) {
295         if (contador_antirebote > tiempo_antirebote) {
296             contador_antirebote = 0;
297             segundos++;
298         } else {
299             contador_antirebote++;
300         }
301     }
```



```
302 } else {  
303     if (!BTN_AJUSTAR_HORA) {  
304         if (contador_antirebote > tiempo_antirebote) {  
305             contador_antirebote = 0;  
306             anio++;  
307             if (anio > 99) {  
308                 siglo++;  
309                 anio = 0;  
310             }  
311         } else {  
312             contador_antirebote++;  
313         }  
314     }  
315  
316     if (!BTN_AJUSTAR_MIN) {  
317         if (contador_antirebote > tiempo_antirebote) {  
318             contador_antirebote = 0;  
319             mes_actual++;  
320         } else {  
321             contador_antirebote++;  
322         }  
323     }  
324  
325     if (!BTN_AJUSTAR_SEG) {  
326         if (contador_antirebote > tiempo_antirebote) {  
327             contador_antirebote = 0;  
328             dia_actual++;  
329         } else {  
330             contador_antirebote++;  
331         }  
332     }
```

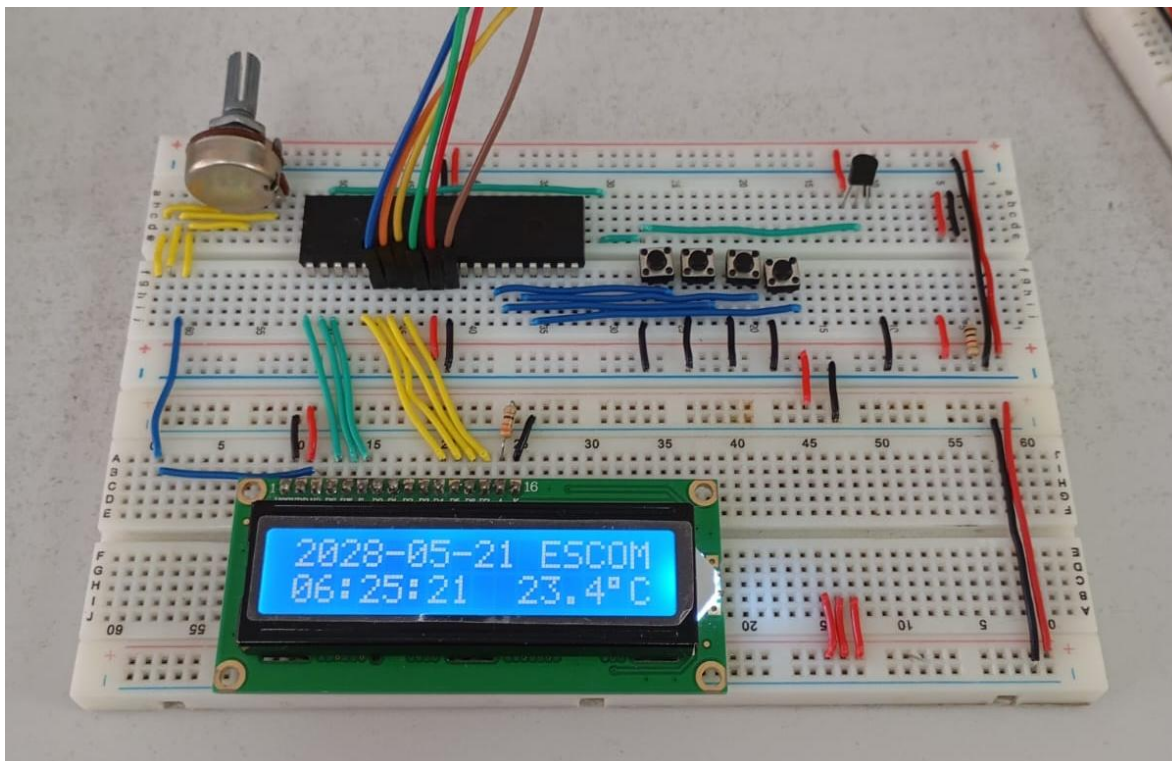


```
333 }  
334  
335 if (contador_reloj > 49) {  
336     contador_reloj = 0;  
337     segundos++;  
338 } else {  
339     contador_reloj++;  
340 }  
341  
342 if (segundos > 59) {  
343     minutos++;  
344     segundos = 0;  
345 }  
346  
347 if (minutos > 59) {  
348     horas++;  
349     minutos = 0;  
350     segundos = 0;  
351 }  
352  
353 if (horas > 23) {  
354     dia_actual++;  
355     horas = 0;  
356     segundos = 0;  
357     minutos = 0;  
358 }  
359  
360 if (dia_actual > 31) {  
361     mes_actual++;  
362     dia_actual = 0;  
363 }
```

```
365     if (mes_actual > 12) {
366         anio++;
367         mes_actual = 0;
368
369         if (anio > 99) {
370             siglo++;
371             anio = 0;
372         }
373     }
374
375     // Colocamos la hora en la LCD
376     lcd_gotoxy(0, 1);
377     lcd_putchar(horas / 10 + ASCII_CERO);
378     lcd_gotoxy(1, 1);
379     lcd_putchar(horas % 10 + ASCII_CERO);
380
381     lcd_gotoxy(2, 1);
382     lcd_putchar(':');
383
384     lcd_gotoxy(3, 1);
385     lcd_putchar(minutos / 10 + ASCII_CERO);
386     lcd_gotoxy(4, 1);
387     lcd_putchar(minutos % 10 + ASCII_CERO);
388
389     lcd_gotoxy(5, 1);
390     lcd_putchar(':');
```

```
392     lcd_gotoxy(6, 1);
393     lcd_putchar(segundos / 10 + ASCII_CERO);
394     lcd_gotoxy(7, 1);
395     lcd_putchar(segundos % 10 + ASCII_CERO);
396
397     // Colocamos la fecha en la LCD
398     lcd_gotoxy(0, 0);
399     lcd_putchar(siglo / 10 + ASCII_CERO);
400     lcd_gotoxy(1, 0);
401     lcd_putchar(siglo % 10 + ASCII_CERO);
402     lcd_gotoxy(2, 0);
403     lcd_putchar(anio / 10 + ASCII_CERO);
404     lcd_gotoxy(3, 0);
405     lcd_putchar(anio % 10 + ASCII_CERO);
406
407     lcd_gotoxy(4, 0);
408     lcd_putchar('-');
409
410     lcd_gotoxy(5, 0);
411     lcd_putchar(mes_actual / 10 + ASCII_CERO);
412     lcd_gotoxy(6, 0);
413     lcd_putchar(mes_actual % 10 + ASCII_CERO);
414
415     lcd_gotoxy(7, 0);
416     lcd_putchar('-');
417
418     lcd_gotoxy(8, 0);
419     lcd_putchar(dia_actual / 10 + ASCII_CERO);
420     lcd_gotoxy(9, 0);
421     lcd_putchar(dia_actual % 10 + ASCII_CERO);
422 }
423 }
```

Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Para el incremento automático de los segundos en el reloj digital lo que se hace es incrementar un contador 50 veces simulando un retardo aproximadamente de un segundo real, dando el efecto de los segundos. Con este incremento automático cuando los segundos llegan a 60 estos se reinician y los minutos incrementan en una unidad, cuando los minutos llegan a 60 estos se reinician y las horas incrementan en una unidad y cuando estas llegan a 24 entonces el reloj se reinicia por completo y en la fecha se incrementan los días en una unidad, dando el efecto de un reloj digital completo.

Para la lectura de la temperatura desde el LM35 lo que se hace es filtrar el ruido que puede presentarse tomando 20 lecturas y calculando el promedio. El resultado de esto será la temperatura que se muestra en la LCD. Como ventaja de esta implementación esta la actualización suave de la temperatura permitiendo su correcta visualización en la LCD.

Para mostrar la temperatura en la LCD al igual que en la práctica del termómetro, esta se divide en decenas, unidades y décimas, las cuales se muestran por separado formando así la temperatura completa.

Por otra parte, tenemos la opción de poder modificar la fecha y la hora por medio de un botón. El modo normal permite modificar por defecto la fecha y el modo edición la hora. Para cambiar entre modos se usa un botón y se detectan los flancos para evitar los rebotes, tal y como se ha hecho en prácticas anteriores. Esto mismo se aplica para los botones para

modificar el día, mes y año de la fecha, así como los segundos, minutos y horas del reloj, respetando por supuesto sus comportamientos.

Por último, se usa la biblioteca `alcd.h` para poder usar la pantalla LCD y sus funciones para mostrar información en ella.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se desarrolló un circuito en el que se destaca el uso de la pantalla LCD 16x2, el cual programamos para que mostrara diferentes elementos dentro del display, a continuación, se mencionan cada uno de ellos y su funcionamiento:

- ✓ **Fecha:** En este primer apartado se muestra la fecha, aquí podemos modificar el año, mes y día de manera independiente gracias a un botón que va incrementando cada valor, para ello hay un botón con el que podemos manipular cada elemento por separado presionando este una vez para cambiar entre año, mes y día. Cada valor reacciona de acuerdo al límite que tienen asignados, por ejemplo, el año no tiene un límite definido, el mes tiene límite hasta 12, y si se incrementa una vez más, el valor regresará a 1 y el valor del año cambiará en consecuencia. Para los días, este tiene un límite de 31, si se excede este límite entonces el valor cambia a 1 y el mes pasara a ser el siguiente.
- ✓ **Palabra:** Este segundo elemento corresponde a las iniciales de la escuela (ESCOM), este apartado no se puede interactuar con él y simplemente muestra la palabra.
- ✓ **Hora:** Este elemento funciona casi de la misma forma que la fecha, pues se pueden editar hora, minutos y segundos con ayuda de los botones. Sin embargo, a diferencia de la fecha, la hora va cambiando en tiempo real con el paso del tiempo con una velocidad parecida a la de un segundo de la vida real. En este caso, si la hora es mayor a 12 el valor se reinicia a 0, así como el de los minutos y segundos. Si los minutos llegan a 60, el valor se reinicia y el valor de la hora incrementa en 1. Y, por último, si los segundos llegan a 60 el valor se reinicia y el valor de los minutos incrementa en 1.
- ✓ **Temperatura:** Finalmente se tiene el valor de la temperatura, este lo programamos de tal forma que toma 20 muestras del LM35 y con ellas calcula un promedio de la temperatura actual, este proceso hace que el cálculo de la temperatura en el display es un poco más lento, pero fuera de eso funciona correctamente tras realizar pruebas con el encendedor.

BIBLIOGRAFÍA

- (s.f.). *Hitachi HD44780 LCD controller*. Wikipedia. Recuperado el 17 de marzo de 2026. https://en.wikipedia.org/wiki/Hitachi_HD44780_LCD_controller
- (s.f.). *Pantalla LCD 16x2*. Luis Llamas. Recuperado el 17 de marzo de 2026. <https://www.luisllamas.es/>